class within the Rails ActiveRecord module and so is available to all classes that subclass ActiveRecord::Base.

This example is still not particularly compelling; it would still be easier to add the log method directly as an instance method of our Logger class. But what happens if we want to construct a different version of the log method for each class that uses it? For example, let's add the capability to add a short class-specific identifying string to the start of each log message. We want to be able to say something like this:

```
class Song < Logger
  add_logging "Song"
end

class Album < Logger
  add_logging "CD"
end
```

To do this, let's define the log method on the fly. We can no longer use a straightforward def…end-style definition—doing so won't allow us to dynamically determine method behavior on the fly. Instead, we'll use one of the cornerstones of Ruby metaprogramming, define_method. This takes the name of a method and a block, defining a method with the given name and with the block as the method body. Any arguments in the block definition become parameters to the method being defined:

metaprogramming/logging_2.rb

```
class Logger
  def self.add_logging(id_string)
    define_method(:log) do |msg|
      now = Time.now.strftime("%H:%M:%S")
      $stderr.puts("#{now}-#{id_string}: #{self} (#{msg})")
    end
  end
end

class Song < Logger
  add_logging "Tune"
end

class Album < Logger
  add_logging "CD"
end

song = Song.new
puts song.log("rock on")
```

produces:

```
00:12:26-Tune: #<Song:0x0000000100c480c8> (rock on)
```

There's an important subtlety in this code. The body of the log method contains this line:

```
$stderr.puts("#{now}-#{id_string}: #{self} (#{msg})")
```

The value now is a local variable, and msg is the parameter to the block. But id_string is the parameter to the enclosing add_logging method. It's accessible inside the block because block definitions create closures, allowing the context in which the block is defined to be carried forward and used when the block is used. In this case, we're taking a value from a class-